Directory Structure - Event Management Dictionary System

Overview

This document describes the file structure for the Event Management dictionary type system.

Directory Tree

```
ZR.Admin.WebApi/Database/
├── hr_init_data.sql                          # HR initialization data
├── hr_schema.sql                             # HR database schema
│
├── sys_dict_type/                            # ← NEW: Dictionary Type
Definitions
│   ├── README.md                             # Documentation for SQL query
patterns
│   ├── STRUCTURE.md                          # This file - directory structure
│   ├── dictionary_queries.sql                # All SQL queries in one file
│   │
│   ├── all_event_management_dict_types.json  # Combined: All 12 dict types
│   │
│   ├── event_type.json                       # Event Types definition
│   ├── zone_type.json                        # Zone Types definition
│   ├── gate_type.json                        # Gate Types definition
│   ├── visitor_type.json                     # Visitor Types definition
│   ├── ticket_type.json                      # Ticket Types definition
│   ├── permit_type.json                      # Permit Types definition
│   ├── booth_type.json                       # Booth Types definition
│   ├── incident_type.json                    # Incident Types definition
│   ├── payment_method.json                   # Payment Methods definition
│   ├── event_status.json                     # Event Status definition
│   ├── event_location.json                   # Event Locations definition
│   └── hr_employee.json                      # HR Employees definition
│
└── SeedData/
    ├── EventManagement/                      # Event Management seed data
    │   ├── README.md
    │   ├── event_types.json                  # Actual event type data
    │   ├── zone_types.json                   # Actual zone type data
    │   ├── gate_types.json                   # Actual gate type data
    │   ├── visitor_types.json                # Actual visitor type data
    │   ├── ticket_types.json                 # Actual ticket type data
    │   ├── permit_types.json                 # Actual permit type data
    │   ├── booth_types.json                  # Actual booth type data
    │   ├── incident_types.json               # Actual incident type data
    │   ├── payment_methods.json              # Actual payment method data
    │   ├── event_status.json                 # Actual event status data
```

```
    │   ├── event_locations.json              # Actual location data
    │   ├── events.json                       # Event records
    │   ├── event_zones.json                  # Zone records
    │   ├── event_gates.json                  # Gate records
    │   ├── event_visitors.json               # Visitor records
    │   ├── event_tickets.json                # Ticket records
    │   ├── event_permits.json                # Permit records
    │   ├── event_booths.json                 # Booth records
    │   ├── event_tenants.json                # Tenant records
    │   ├── event_contracts.json              # Contract records
    │   ├── event_incidents.json              # Incident records
    │   ├── event_payments.json               # Payment records
    │   └── event_invoices.json               # Invoice records
    │
    └── sys_dict_type/
        └── EventManagement/                  # ← Generated dictionary data
            ├── sys_dict_type_event_management.json
            ├── dict_data_event_types.json
            ├── dict_data_zone_types.json
            ├── dict_data_gate_types.json
            ├── dict_data_visitor_types.json
            ├── dict_data_ticket_types.json
            ├── dict_data_permit_types.json
            ├── dict_data_booth_types.json
            ├── dict_data_incident_types.json
            ├── dict_data_payment_methods.json
            ├── dict_data_event_status.json
            ├── dict_data_event_locations.json
            ├── dict_data_hr_employees.json
            └── dict_data_all_combined.json
```

File Categories

1. Dictionary Type Definitions (`sys_dict_type/`)

Purpose: Define the dictionary types that will be used in the application

Files:

- Individual JSON files for each dictionary type
- Combined JSON file with all types
- SQL query reference file
- Documentation

Usage:

- Manual import into `sys_dict_type` table
- Reference for understanding the dictionary structure
- Template for adding new dictionary types

2. Source Data (`SeedData/EventManagement/`)

Purpose: Source data for Event Management entities

Contains:

- Raw data for all Event Management tables
- Type definitions (event_types, zone_types, etc.)
- Entity records (events, tickets, permits, etc.)

Generated By:

- Manual data entry or Excel import
- Database exports

3. Generated Dictionary Data (`SeedData/sys_dict_type/EventManagement/`)

Purpose: Auto-generated dictionary data from source tables

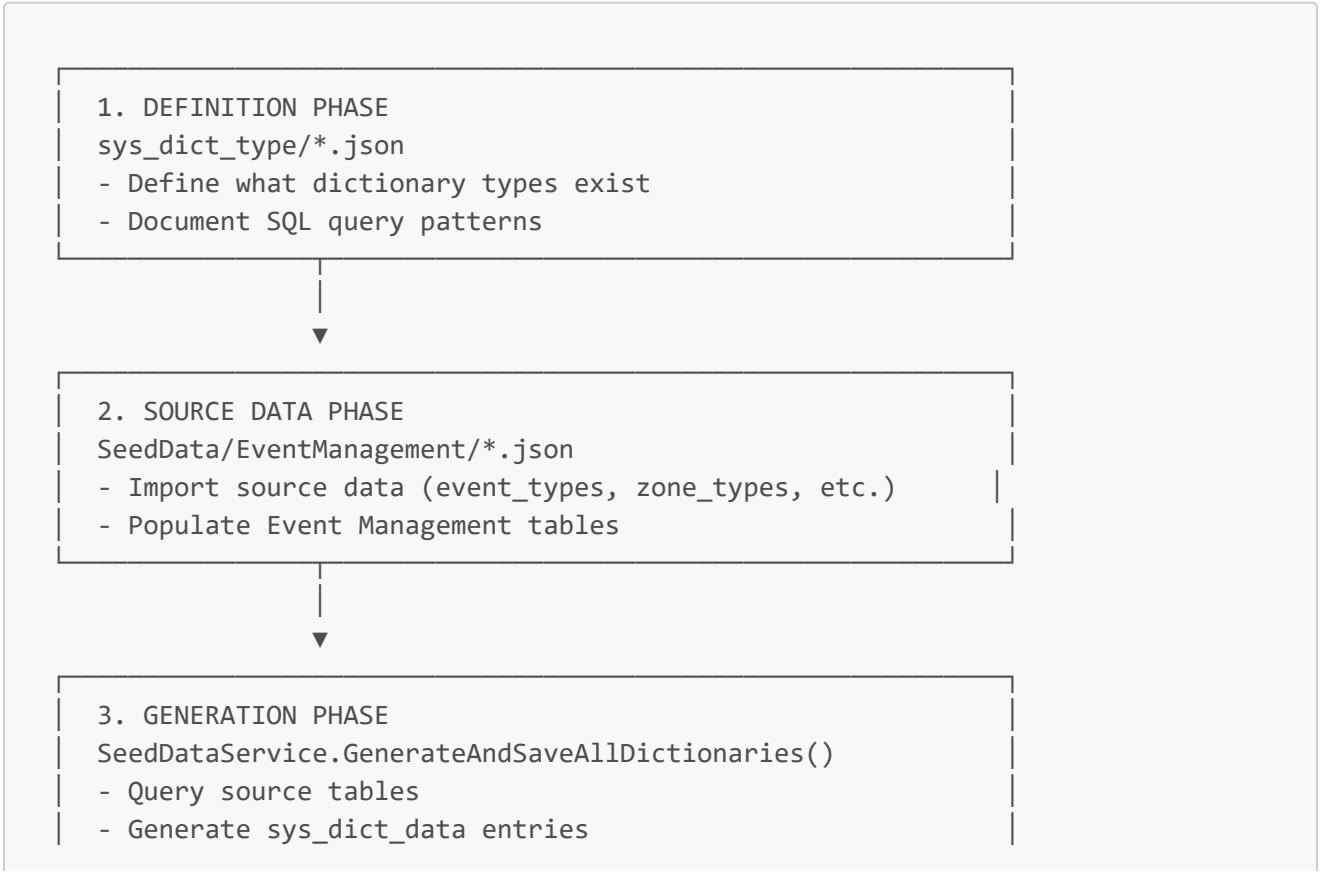
Contains:

- `sys_dict_type` entries
- `sys_dict_data` entries for each type
- Combined data files

Generated By:

- `SeedDataService.GenerateAndSaveAllDictionaries()`
- Runs automatically during seed data initialization

Data Flow



```
│  - Save to SeedData/sys_dict_type/EventManagement/          │
└──────────────────────┬──────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  4. DATABASE PHASE                                           │
│  - Insert/Update sys_dict_type table                         │
│  - Insert/Update sys_dict_data table                         │
│  - Dictionary ready for use in dropdowns/selects             │
└──────────────────────────────────────────────────────────────┘
```

SQL Query Pattern

All dictionary types follow this pattern (based on HR employee example):

```sql
SELECT
    [primary_key] AS dictValue,
    CONCAT([field1], ' - ', [field2], ' - ', [field3]) AS dictLabel,
    remark
FROM [source_table]
ORDER BY [primary_key];
```

Examples:

Event Types:

```sql
SELECT typecode AS dictValue,
       CONCAT(typecode, ' - ', typename, ' - ', typenamear) AS dictLabel,
       remark
FROM event_types;
```

HR Employees:

```sql
SELECT employeeid AS dictValue,
       CONCAT(firstname, ' ', lastname, ' ', namear) AS dictLabel,
       remark
FROM hr_employee;
```

Event Locations:

```sql
SELECT locationid AS dictValue,
       CONCAT(locationid, ' - ', locationname, ' - ', city) AS dictLabel,
       address AS remark
FROM event_locations;
```

Key Features

1. Bilingual Support

All labels include both English and Arabic text:

- `typename` (English) + `typenamear` (Arabic)
- Example: "VIP - VIP Guest - ضيف كبار الشخصيات"

2. Automatic Updates

When source data changes:

1. Re-run seed data initialization
2. System automatically regenerates dictionary data
3. Updates both JSON files and database

3. Flexible Structure

- Individual files for granular control
- Combined files for bulk operations
- SQL files for direct database operations

Implementation Details

C# Code Location: `ZR.ServiceCore/Services/SeedDataService.cs`

Key Methods:

- `GenerateEventManagementDictTypes()` - Lines 1346-1449
- `GenerateDictDataFrom[Type]()` - Lines 1451-1725
- `GenerateAndSaveAllDictionaries()` - Lines 1754-1845

Database Tables:

- `sys_dict_type` - Dictionary type definitions
- `sys_dict_data` - Dictionary data entries

Usage Examples

Import Dictionary Types

```
# Import all dictionary types
POST /api/system/seed/initialize

# Or use JSON files directly
psql -d MahamDB -c "COPY sys_dict_type FROM
'/path/to/all_event_management_dict_types.json';"
```

Query Dictionary Data

```
// Get all event types for dropdown
var eventTypes = await _dictDataService
    .GetDictDataByType("event_type");

// Get HR employees for assignment
var employees = await _dictDataService
    .GetDictDataByType("hr_employee");
```

Add New Dictionary Type

1. Create JSON definition in `sys_dict_type/[new_type].json`
2. Add to `all_event_management_dict_types.json`
3. Add method in `SeedDataService.cs`:

    ```
    public List<SysDictData> GenerateDictDataFrom[NewType]()
    {
        // Query source table
        // Generate dict data entries
        // Return list
    }
    ```

4. Add to `GenerateAndSaveAllDictionaries()` method
5. Re-run seed initialization

Maintenance

Adding New Types

1. Follow naming convention: `[entity]_type` or `[module]_[entity]`
2. Update `all_event_management_dict_types.json`
3. Add SQL query to `dictionary_queries.sql`
4. Implement generation method in C#

Updating Existing Types

1. Modify source data in `SeedData/EventManagement/`
2. Re-run seed initialization
3. Generated files automatically update

Troubleshooting

- Check logs in seed initialization results
- Verify source tables have data

- Ensure column names match expected pattern
- Check file permissions for JSON output

Related Documentation

- `README.md` - SQL query patterns and usage
- `dictionary_queries.sql` - All SQL queries
- `SeedData/EventManagement/README.md` - Source data documentation
- `ZR.ServiceCore/Services/SeedDataService.cs` - Implementation code